2026 全国大学生 ICT 大赛 • 网信产业应用赛道 • 赛题九

智能体驱动的 AI 安全知识情报系统

技 术 报 告

情报监测 • 知识富化 • 证据问答 • 处置闭环 的一体化工作台

作品名称 智能体驱动的 AI 安全知识情报系统

参赛赛题 赛题九 网信产业应用赛道

交付形态 本地单机 Web 工作台 • Docker 镜像 • 持续集成流水线

演示机快照 2026 年 10 月 18 日（705 条事件 / 1059 份文档）

重要内容预览

一页读懂：这套系统解决什么问题、凭什么可信、交付到什么程度

本系统面向 AI 安全领域，把分散在漏洞库、厂商公告、学术论文、标准框架与安全社区里的公开情报，整理成一条可持续运行的证据链：**自动采集 → 结构化富化 → 可追溯问答 → 处置闭环**。它不生成没有出处的结论，而是把每一条判断钉回到原文的具体字符位置。

四个核心亮点

01 证据优先的答案组织

答案先给结论句，再附逐字原文与 chunk 编号；引用取不回就不写进答案。同一批 50 题实测引用可追溯 100%。

02 可复算的检索与判定

关键词与同义词双路召回后统一打分，分数可复算；机器指标与人工指标分列，不混算成一个好看的数字。

03 可审计的处置闭环

处置记录带处置人、版本前后与复测结论；关闭只能由系统复测确认，接受风险单独计数、不冒充风险消除。

04 无人值守也能自愈

内置调度器每小时运行；连续 3 次失败或陈旧的来源自动降级冷却 6 小时后重试，历史数据保留并标注 stale。

交付时的实测读数

表 1　交付核心指标一览（快照 2026-10-18）

维度	指标	读数	口径来源
语料	入库事件 / 关联文档	705 条 / 1059 份	知识库概览接口
监测	漏洞情报采集时延中位数	7.34 小时（n=244）	采集时间戳与发布时间
监测	24 小时内发现比例	73.8%	同上
问答	文档命中 / 要点命中	97.22% / 92.11%	固定 50 题评测集
问答	引用可追溯 / 拒答一致	100% / 84%	引用 ID 回库校验
关系	精确率 / 召回率 / F1	1.000 / 0.734 / 0.8466	人工金标准 94 条
处置	高优先级闭环率	96.36%（53/55）	处置状态表
工程	自动化测试	750 项通过 / 0 跳过	pytest 全量运行

阅读提示　第 2 至 7 章讲设计，第 8 章讲选型理由，第 9 章讲实现细节，第 10 至 11 章给效果与实测，第 12 章收束创新点与边界。附录给出全部指标口径与复现步骤，任何数字都可以按附录重跑。

目 录

第 1 章 概述 6
1.1 赛题背景与问题定义 6
1.2 设计目标 6
1.3 系统边界与约束 6
1.4 术语与口径约定 7
第 2 章 系统总体架构设计 7
2.1 架构原则 7
2.2 分层结构 7
2.3 组件职责 9
2.4 部署形态与运行方式 9
第 3 章 智能体方案设计 10
3.1 角色划分与职责边界 10
3.2 任务 DAG 与工具预算 12
3.3 模型调用与降级链 12
3.4 证据约束下的答案生成 13
3.5 多 Agent 协作与审核 13
第 4 章 情报监测模块设计 13
4.1 来源体系 13
4.2 采集流水线 14
4.3 归一化与去重 16
4.4 调度与自愈 16
4.5 运行可观测性 18
第 5 章 知识富化模块设计 18
5.1 富化维度 18
5.2 CVSS 与版本区间 19
5.3 POC 与论文关联 19

5.4 证据型关系抽取 19
第 6 章 知识库与证据问答 20
6.1 知识组织 20
6.2 检索与打分 20
6.3 引用定位 22
6.4 拒答与澄清 22
6.5 问答时序 22
第 7 章 处置闭环设计 24
7.1 状态机 24
7.2 复测机制 26
7.3 口径与统计 26
第 8 章 关键技术选型说明 26
8.1 选型总览 26
8.2 为什么用 SQLite 27
8.3 为什么不用向量检索 27
8.4 模型为何可插拔 27
8.5 前端为何不用框架 27
第 9 章 系统实现细节 27
9.1 工程结构 27
9.2 数据模型 28
9.3 核心代码说明 30
9.4 接口设计 31
9.5 一致性与并发 32
第 10 章 功能模块实现效果 32
10.1 任务工作台 32
10.2 知识图谱 33
10.3 赛题指标 33
10.4 模块效果对照 34

第 11 章　性能测试结果与分析 34
11.1 测试环境与方法 34
11.2 采集时效 35
11.3 证据问答质量 35
11.4 关系抽取评测 36
11.5 工程验证 36
11.6 结果分析 37
第 12 章　创新点与展望 37
12.1 创新点 37
12.2 局限 37
12.3 展望 38
附录 A　指标口径 38
附录 B　复现步骤 38

第 1 章　概述

1.1 赛题背景与问题定义

AI 安全已经不再是单一产品的附加功能。模型、智能体、插件与工具调用共同构成新的攻击面，相关情报同时散落在国家漏洞库、厂商公告、学术预印本、行业标准与安全社区里。对一线安全团队而言，真正困难的不是“找不到信息”，而是三件事：**信息来得太散、结论落不了地、出了问题说不清依据**。

赛题要求构建一个由智能体驱动的 AI 安全知识情报系统，覆盖情报监测、知识富化、证据问答与处置闭环。我们把这句话拆成四个可验证的问题：系统能不能持续跑下去；抓回来的东西能不能变成结构化的知识；回答问题时能不能指出依据；给出处置建议后能不能形成闭环并被复查。本报告的全部设计，都是围绕这四个问题展开的。

1.2 设计目标

系统在设计阶段确立了四条硬约束，后续所有取舍都以此为准绳。

- **可持续**：无人值守运行，单个来源失败不能拖垮整批采集，失败要有降级、有冷却、有恢复。
- **可追溯**：任何一条结论都必须能回到某个来源、某份文档、某个字符区间；取不回来就拒绝作答。
- **可评测**：指标从接口和数据库直接读出，不靠人工填报；机器指标与人工指标分开列，不合成单一分数。
- **可交付**：单机一条命令可运行，容器可构建，关键路径有自动化测试与持续集成证据。

1.3 系统边界与约束

为了避免把演示做成“看起来什么都会”，系统明确划定了边界，并在页面与报告里如实标注。

表 1-1　系统边界与对外表述

边界	系统做法	对外表述
数据来源	只采集登记在册的 20 个公开来源，不临时拼接外部地址	公开情报，非暗网或付费库
资产数据	仓库内置 8 条演示资产，明确标注 is_demo	合成演示资产，不作为真实企业资产
漏洞验证	只记录公开 POC 记录与修复版本，不执行利用	不做漏洞利用，不提供攻击工具
模型能力	模型只用于受限规划与措辞改写，事实由本地证据决定	无大模型时系统仍可完整作答
决策权	系统给建议，处置动作由人确认并留痕	辅助决策，不替代安全负责人

1.4 术语与口径约定

报告中反复出现的几个词有严格定义，后文的指标都按这里的口径统计，避免同一名词在不同章节指代不同东西。

表 1-2 术语定义

术语	定义
事件（event）	一条来自外部来源的公开情报记录，可能是漏洞、论文、标准或政策
文档（document）	事件的原文载体；同一事件可能关联多份文档
分块（chunk）	文档按章节切分后的最小检索单元，记录 chunk_id、字符起止与内容哈希
证据（evidence）	可被引用的来源记录；引用必须能按 ID 回库取回
研判（assessment）	系统对“某事件是否影响某资产”的结构化判断，结论为受影响 / 不受影响 / 待确认
处置（disposition）	人对研判结论采取的动作及结果，带处置人、时间与复测记录
降级（degradation）	来源连续失败后退出无人值守集合，冷却后自动重试的机制

第 2 章 系统总体架构设计

2.1 架构原则

系统采用单进程、单数据库的紧凑架构。这个选择不是能力不足的妥协，而是针对赛题场景的主动取舍：评审需要在一台普通笔记本上复现全部能力，任何需要额外中间件的设计都会抬高复现成本，也会引入新的失败点。

在紧凑形态下，我们坚持四条原则。**第一，确定性优先**：能用代码算准的事情不交给模型，版本区间比较、去重、打分、状态流转全部是确定性逻辑。**第二，证据优先**：数据表里同时存事实与判断，事实来自外部来源，判断由系统生成，两者不混在一张表里。**第三，失败可见**：抓不到就记失败，缺数据就留空，不用估计值补齐。**第四，单库闭环**：采集、富化、检索、处置共用一份 SQLite，任何环节都能顺着主键回溯。

2.2 分层结构

系统自上而下分为接入层、编排层、能力层与数据层。接入层是浏览器里的单页工作台，编排层是 FastAPI 提供的接口与任务调度，能力层由采集、归一、富化、检索、研判、处置六个模块组成，数据层是一份 SQLite 数据库与按内容哈希落盘的原文快照。整体数据流如图 1 所示。

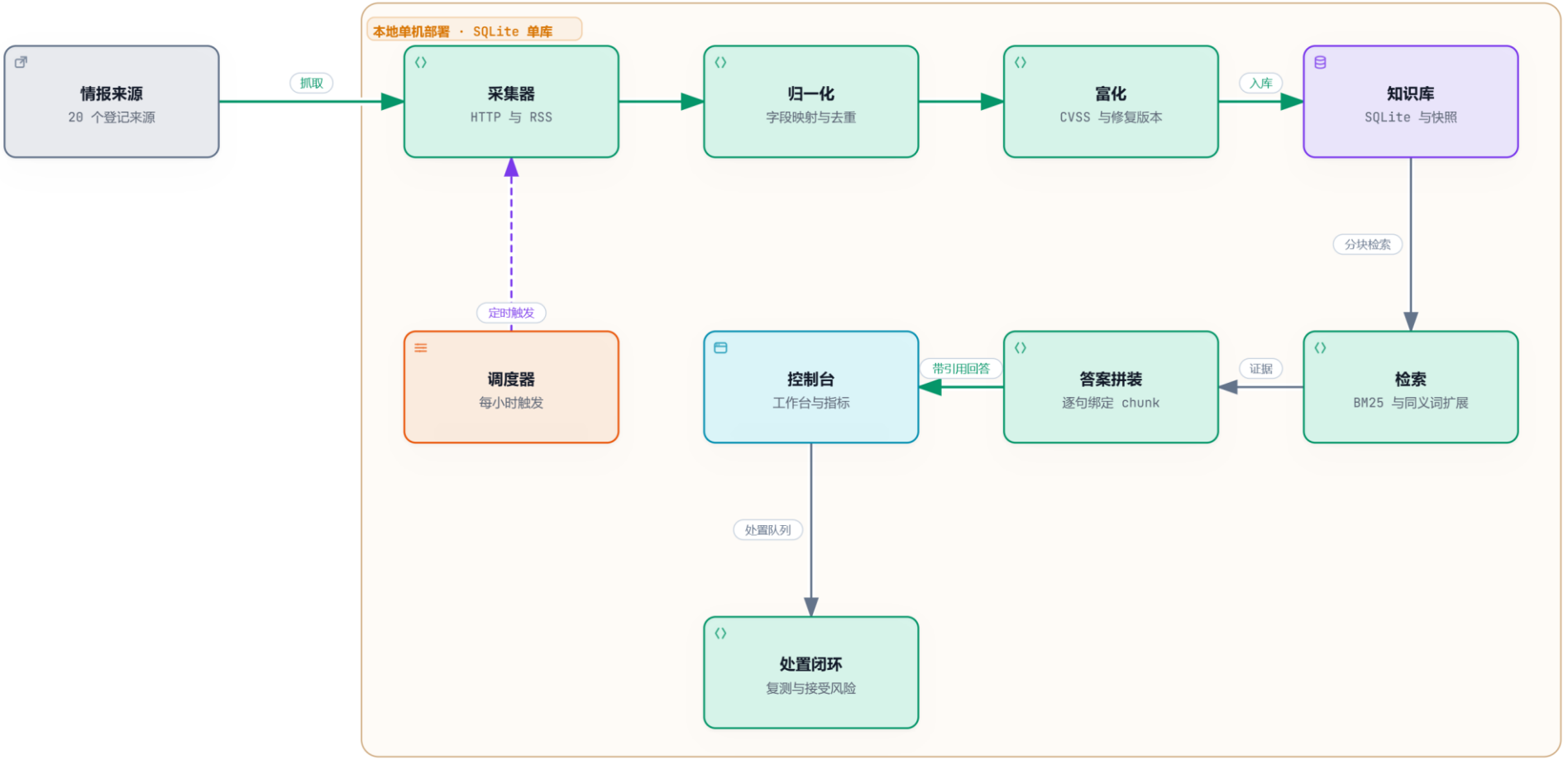


图 1　系统总体架构与主数据流

图中橙色虚线框标出的是本地单机部署边界：采集器、归一化、富化、知识库、检索、答案拼装与处置闭环全部在同一个进程内完成，不依赖外部消息队列或向量数据库。唯一的外部依赖是左侧登记的 20 个情报来源，它们通过 HTTP 接口或 RSS 订阅进入系统，原文按 SHA-256 落盘后进入归一化流程。

2.3 组件职责

每个组件只做一件事，并且对外暴露稳定的输入输出。这样既方便单测，也让“哪一层出了问题”变得可判断。

表 2-1　组件职责划分

组件	职责	关键实现
工作台前端	三页签单页应用：任务工作台、知识图谱、赛题指标	原生 HTML/CSS/JS，无构建链
接口层	参数校验、编排调用、SSE 流式输出、跨站写保护	FastAPI + Pydantic 严格模型
采集器	按来源定义抓取、单源预算、失败分类	HTTP / RSS / PDF 三通道
归一化	字段映射、时间解析、去重与别名归并	确定性规则 + 内容哈希
富化	CVSS、版本区间、修复版本、POC、论文关联、关系抽取	规则优先，缺口如实登记
知识库	事件、文档、分块、术语与关系存储	SQLite 单库 + 快照
检索与作答	双路召回、融合打分、引用定位、拒答判定	BM25 + 术语扩展 + 邻接补齐
智能体编排	任务 DAG、角色交接、工具预算、检查点	内置编排器 + 多 Agent 运行时
自愈与调度	定时触发、失败重试、降级冷却、告警	来源级状态机
处置闭环	处置状态、版本前后、复测结论与指标	状态机 + 证据绑定

2.4 部署形态与运行方式

系统提供三种运行形态，覆盖从个人演示到流水线验证的不同需求。三种形态共用同一份代码与同一套数据目录约定，不存在“演示版”和“正式版”两套实现。

表 2-2　三种部署形态

形态	启动方式	适用场景	验证证据
本地直跑	python -m uvicorn app.api:app --port 8000	现场演示、开发调试	演示机连续运行 24 个采集日
容器运行	docker compose up	环境一致性验证	CI 容器作业构建并启动健康检查
持续集成	GitHub Actions	每次提交的回归验证	容器 / 回滚 / 测试三项作业全绿

容器镜像基于 python:3.12-slim，只安装运行期依赖，暴露 8000 端口并内置健康检查，检查的目标就是 /api/health。持续集成流水线里除了测试，还包含一次容器构建与一次回滚验证，后者用于确认数据库备份与恢复路径可用。

第 3 章　智能体方案设计

3.1 角色划分与职责边界

系统里有六个执行角色：规划器、采集员、归一化、检索员、审核员与调度员，另有一个可选的答案组织角色，只在启用模型改写时参与。把角色拆开的目的不是凑数量，而是让每一段工作都有明确的输入、输出与失败处理方式。图 2 给出角色之间的协作关系与职责边界。

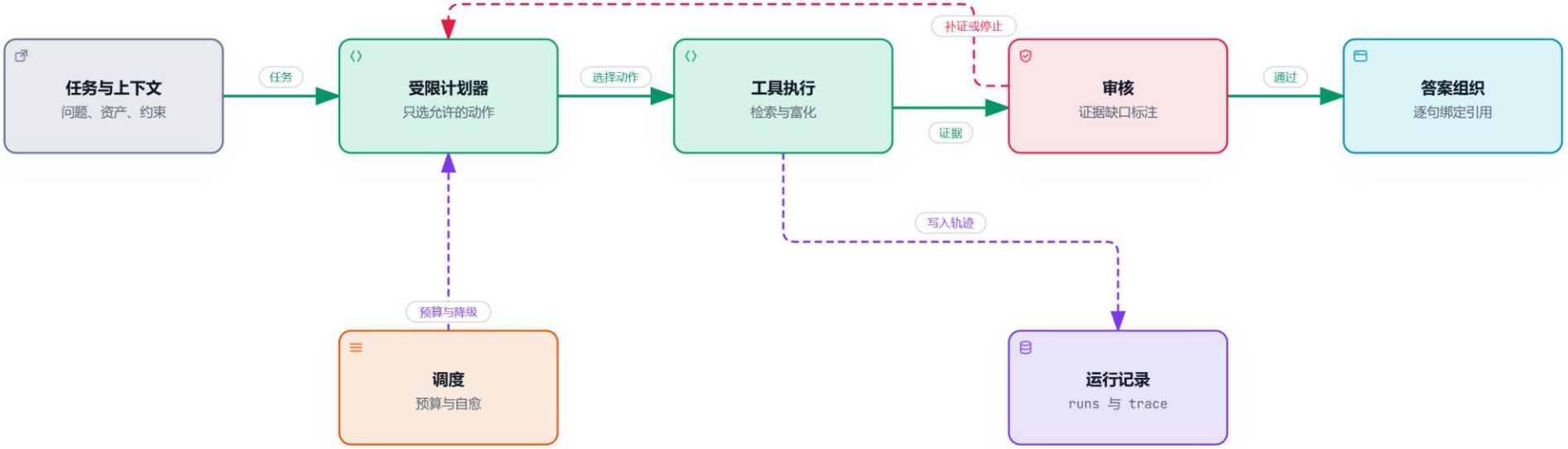


图 2　智能体角色编排与工具边界

这七个角色绑定里只有两个允许接触模型：规划器用模型在受限动作表里做选择，答案组织可选地用模型改写措辞。采集、归一化、检索、审核与调度全部由确定性代码完成。这条边界的作用是：**模型可以决定“先做什么”，但不能决定“事实是什么”**。即使模型完全不可用，系统仍能按本地模板走完整个流程并给出带引用的答案。

3.2 任务 DAG 与工具预算

一次完整的研判任务被建模为有向无环图，节点之间用依赖关系约束执行顺序，每个节点带成功判据与工具预算。预算的作用是防止单个环节无限重试，也让“这一轮到底调用了多少次工具”可以被审计。

表 3-1　任务 DAG 的节点与预算

节点	角色	动作	成功判据	预算
collect	采集员	按登记来源抓取	来源返回并记录状态	单源一次
normalize	归一化	字段映射与去重	生成标准事件且无重复	一次
enrich	检索员	补齐 CVSS / 版本 / POC	缺口被记录或已补齐	最多 6 条
assess	审核员	资产影响研判	每条结论带证据编号	一次
answer	答案组织	生成带引用的答案	引用可回库取回	一次

编排器在节点之间传递的是结构化结果而不是自然语言，这让后续节点可以精确判断上一节点是否真的完成。每个节点执行后写入检查点，任务被中断时可以从最近的检查点恢复，而不是整轮重来。

3.3 模型调用与降级链

系统把模型调用设计成三级降级链：优先使用配置好的远端模型；模型不可用时回落到本地受限规划模板；两者都不可用时直接使用纯本地检索链路。需要说明的是，**第三级之所以存在，是因为本系统没有部署本地模型运行时**，而不是为了凑一个“本地层”的说法。这一点在接口返回与页面展示里都写明了。

表 3-2　模型降级链

层级	触发条件	行为	对外标注
一级	已配置模型密钥且连通	模型在受限动作表内做规划	model_assisted
二级	模型不可用或返回非法结构	回落本地规划模板，记录降级原因	local_evidence_extraction
三级	无本地模型运行时	纯本地检索与规则作答	明确标注“无本地模型”

降级不是静默失败。每次降级都会在响应里留下 mode 字段与原因，页面顶部会显示当前处于哪种模式。这样评审看到的每一种答案，都能知道它是谁生成的、依据是什么。

3.4 证据约束下的答案生成

答案生成的规则可以概括为一句话：**先有证据，再有句子**。系统先检索出候选事件与文档分块，再按固定模板拼装结论句，每句话都绑定一个或多个证据编号。如果某个问题在证据里找不到支撑，系统不会“顺着说下去”，而是明确拒答并给出原因。

这套约束带来一个直接好处：答案可以被机器复核。评测脚本逐条检查引用 ID 能否回库取回、结论句里的术语是否在原文中出现，从而把“答得好不好”拆成可测量的几个维度。第 11 章给出的问答指标，就是这样算出来的。

3.5 多 Agent 协作与审核

在单进程编排之上，系统另外实现了多 Agent 运行时：每个 Agent 通过消息总线收发结构化消息，交接需要携带证据引用，审核节点可以驳回并触发重规划。冲突仲裁遵循“证据多者优先、时间新者优先、无法判定则挂起待人工确认”的规则，避免系统在没有依据时硬给结论。

多 Agent 运行时的每次执行都会落一条运行记录，包含参与角色、消息条数、工具调用次数与最终状态。这让“系统到底做了什么”不只停留在日志里，而是可以被页面查询与复现。

第 4 章　情报监测模块设计

4.1 来源体系

监测能力取决于来源结构。系统登记了 20 个公开来源，按业务视角归为 6 组，覆盖权威漏洞库、在野利用、厂商公告、学术论文、标准框架、政策法规与安全社区。页面上的一级筛选则合并为 4 类（漏洞、论文、标准与框架、政策法规），两者是同一份登记表的不同视角，不存在两套来源清单。

表 4-1　20 个登记来源的分组

来源分组	代表来源	数量	采集方式
权威漏洞库	NVD、MITRE CVE、OSV、GitHub Advisory	4	接口
在野利用与厂商公告	CISA KEV、Microsoft MSRC	2	订阅源 / 接口
学术论文	arXiv（cs.CR / cs.LG）、OpenAlex	2	接口
标准与风险框架	OWASP GenAI、NIST AI RMF、MITRE ATLAS	3	页面解析
政策法规与标准动态	EU AI Act、NIST News	2	PDF / 订阅源
安全博客与社区	FreeBuf、安全客、补天、Securelist、Talos、奇安信威胁情报、GitHub Security Blog	7	订阅源 / 页面

每个来源在登记表里带有类别、采集方式、信任级别、许可说明与回溯窗口。信任级别决定了它在证据链里的权重，回溯窗口则解决低频来源“首次采集就跳过最新内容”的问题——月度更新的标准页面如果只回溯 30 天，很容易在没有游标的情况下漏掉刚发布的版本。

截至快照时点，20 个来源中有 15 个在本周期产出了数据。剩余 5 个并非故障：它们本身不是 AI 安全专源，抓取成功但内容未通过相关性判定，运行记录里如实标注了“抓取成功、过滤为 0”。对外表述统一为“15 个来源持续产出”，不把“登记了 20 个”说成“20 个都在产出”。

4.2 采集流水线

一次采集从读取登记表开始，到事件入库与运行留痕结束，中间经过抓取、归一、去重与相关性过滤四个环节。流水线的设计要点是每一步都留下可复核的证据：原始响应、标准字段、过滤理由与写入计数。

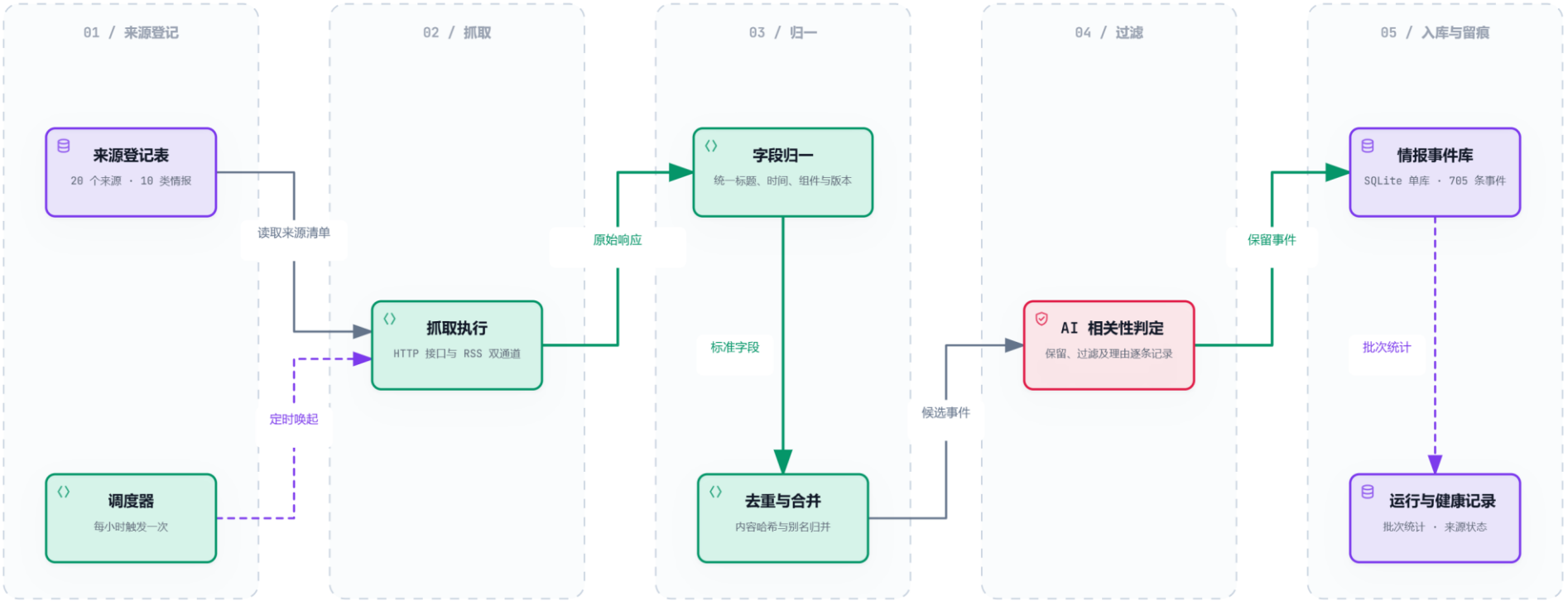


图 3　情报采集与归一流水线

抓取环节按来源类型选择通道：结构化接口直接返回 JSON，订阅源返回 RSS/Atom，标准与法规页面走 HTML 或 PDF 解析。每个来源有独立的抓取预算，单个来源超时或限流只会让这一路失败，不会阻塞整批。抓取结果先落原文快照，再做解析，这样即使解析逻辑更新，也能用原始字节重新跑一遍。

相关性判定是这条流水线上唯一使用模型的环节，判定结果是“保留 / 过滤”与一段理由。理由会随事件一起存入数据库，评审在知识图谱页面可以逐条查看“这条为什么被判为不相关”。把过滤理由留下来，是这套系统与“抓完就丢”的采集脚本最本质的区别。

4.3 归一化与去重

不同来源的字段命名差异极大：NVD 用 `published`，RSS 用 `pubDate`，CVE 记录把受影响版本放在嵌套数组里。归一化层把这些差异收敛成统一结构，包括标题、组件、生态、发布时间、受影响范围、修复版本、参考链接与严重性。所有映射规则都是显式代码，没有隐式猜测。

去重采用两级策略。第一级是内容哈希：同一份原文无论来自哪个渠道，哈希一致即视为重复。第二级是别名归并：CVE 编号、GHSA 编号与厂商内部编号通过别名表关联到同一事件，这样“同一条漏洞被三个库分别收录”不会变成三条独立情报，但三个来源都会保留在证据链里。

4.4 调度与自愈

系统内置调度器，每小时触发一次采集，不依赖外部 cron。调度器面临的真实问题是：外部来源会限流、会超时、会临时下线，如果每次都按同等力度重试，既浪费预算又会放大故障。因此我们把来源健康度建模成状态机，如图 4 所示。

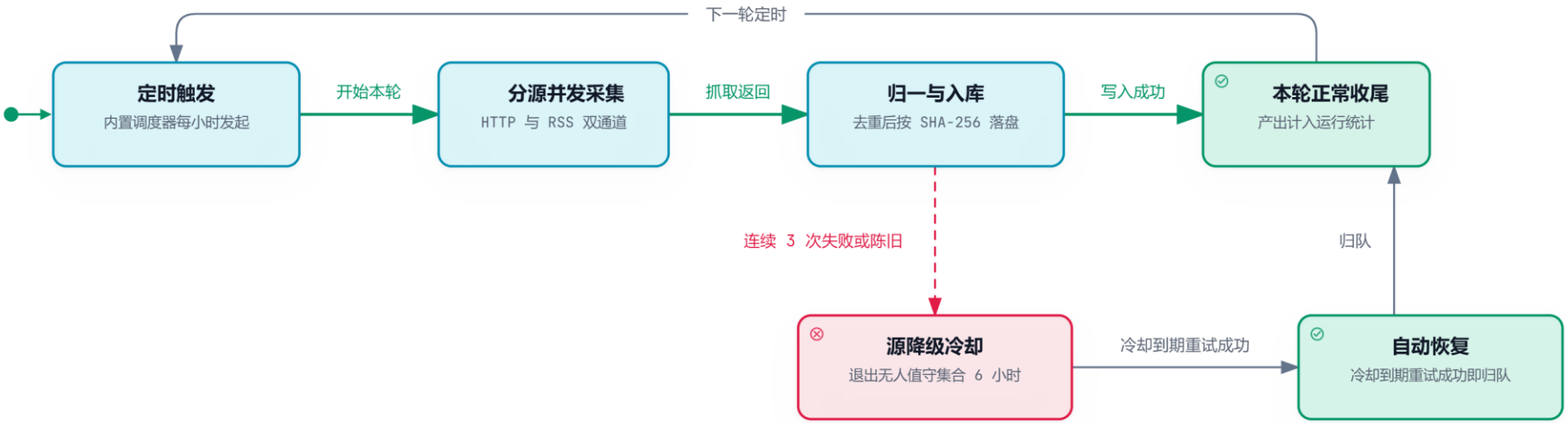

下一轮定时
定时触发
内置调度器每小时发起
开始本轮
分源并发采集
HTTP 与 RSS 双通道
抓取返回
归一与入库
去重后按 SHA-256 落盘
写入成功
本轮正常收尾
产出计入运行统计
连续 3 次失败或陈旧
归队
源降级冷却
退出无人值守集合 6 小时
冷却到期重试成功
自动恢复
冷却到期重试成功即归队

图 4　调度与自愈状态机

判定口径经过一次修订：**有产出的 partial 不算失败**。GitHub Advisory 触及抓取预算是采集器的设计行为，只要它仍然返回了数据，就不应该被计为故障；只有“跑得起来但一条都没采到”的 partial 才计入连续失败。stale 与 failed 同等计入，因为它意味着这一轮确实没有拿到新数据。

连续 3 次失败或陈旧后，来源退出无人值守集合，进入 6 小时冷却；冷却到期后系统重新探测，成功即自动归队。降级期间历史数据不会被清空，页面把该来源标注为 stale，读者能看出“这是旧数据”而不是“没有数据”。每次降级都会记录 failure_class、last_error 与当时的来源状态，事后可以逐条复核。

4.5 运行可观测性

系统把每一次运行都写成一条记录，包含类型、状态、起止时间、摘要与明细。演示机上累计 1339 条运行记录，覆盖采集、富化、研判、问答与评测五类动作。这些记录不是装饰：赛题指标页的“连续运行天数”只统计 trigger=scheduled 的运行，手工点击产生的记录不会被算进去。

来源健康面板展示每个来源的最近采集时间、成功/失败状态、连续失败次数与降级剩余时间。系统健康接口另外汇总四个模块的状态：采集与监测、自愈与降级、知识库与问答、告警。页面上的“需关注”不是情绪化提示，而是最近 20 条告警里存在 warning 及以上的明确判据。

第 5 章　知识富化模块设计

5.1 富化维度

原始情报只是“一条链接加一段摘要”，要变成可研判的知识，需要补齐六个维度：CVSS 评分、受影响版本区间、修复版本、POC 记录、论文关联与证据型关系。富化模块按维度逐个尝试，能补则补，补不上就登记缺口。

表 5-1　六个富化维度

维度	实现方式	写入位置	已富化事件
CVSS 评分	从 NVD / OSV / 厂商公告解析向量与分值	事件文档的 cvss 数组	480
受影响版本区间	解析 affected 结构，统一为语义化区间	事件文档的 affected	110 条含修复版本
修复版本	从公告正文与结构化字段提取	affected[].fixed_version	110
POC 记录	识别公开 POC 链接与验证状态	事件文档的 poc	96
论文关联	识别论文类事件与 arXiv 标识，建立事件到论文的边	relationships	162
证据型关系	抽取并绑定 evidence_ids 的显式关系	relationships	38 条关系

表里的“已富化事件”是绝对数量，不是覆盖率。分母始终是全部 705 条事件，但很多维度只对漏洞类事件有意义，论文与政策类事件天然没有 CVSS。因此系统不把“覆盖率低”包装成“富化失败”，而是按类别分别统计，并在缺口清单里写清楚缺的是哪一项。

5.2 CVSS 与版本区间

版本判断是资产研判里最容易出错的一环。字符串比较会把 1.9 判成大于 1.10，而真实语义版本恰好相反。系统统一使用语义化版本比较，并支持区间表达式的解析，无法解析的区间一律落到“待确认”，绝不默认判为安全。

这条规则看起来保守，但它决定了系统的可信度：在安全研判里，“不确定”必须显式表达为不确定，而不是被默认成“没事”。页面上每一条待确认结论都附带原因，例如“事件缺少该组件的可解析受影响版本范围”。

5.3 POC 与论文关联

POC 记录只登记公开可查的链接与验证状态，不执行任何利用代码。系统区分“有公开 POC”“仅有传闻”“未收录”三种状态，并在页面上如实展示——没有收录不等于不存在，这一点在口径文档里有明确说明。

论文关联面向 AI 安全的研究脉络：系统识别论文类事件，提取 arXiv 标识，把论文与相关漏洞事件连成边。这样在做技术汇报时，可以从一个漏洞追到它背后的攻击方法研究，而不是只看到一个编号。

5.4 证据型关系抽取

关系抽取的目标不是“画一张好看的图”，而是回答“这两条信息之间凭什么有关系”。系统只保留带 evidence_ids 的关系，每条边都能指回具体的来源与字符区间。评测采用人工金标准，逐条判定抽取是否正确。

表 5-2　证据型关系的类型与判据

关系类型	示例	证据要求
影响关系	CVE → 受影响组件与版本区间	公告中的 affected 结构
修复关系	CVE → 修复版本	公告正文或字段中的 fixed_version
论文关联	CVE → 研究论文	论文摘要或正文中的明确引用
文档引用	论文 A → 论文 B	参考文献条目与字符偏移
术语归属	术语 → 相关事件	术语在原文中的逐字出现

当前批次的人工评测结果是精确率 1.000、召回率 0.734、F1 0.8466，TP 69 / FP 0 / FN 25。精确率为 1 意味着系统没有生成任何一条无证据的关系边；召回率的缺口主要来自语料本身，部分文档之间确实不存在可核验的引用关系，这部分被如实排除在分母之外，而不是用推测补齐。

第 6 章　知识库与证据问答

6.1 知识组织

知识库不是一堆文档的堆叠，而是四层结构：事件层记录“发生了什么”，文档层保存原文，分块层提供可检索的最小单元，术语层把领域词汇与证据绑定。四层之间用稳定主键关联，任何一层都能回溯到相邻层。

表 6-1　知识库的四层结构

层次	内容	规模（快照）	用途
事件层	结构化情报记录与富化字段	705 条	检索、研判、统计
文档层	原文快照与元数据	1059 份	引用与复核
分块层	按章节切分的检索单元	849 个分块	召回与引用定位
术语层	AI 安全术语与证据绑定	26 个术语（覆盖数以接口为准）	查询扩展与解释

分块记录 chunk_id、字符起止、所属章节与内容哈希。字符偏移是引用可追溯的关键：读者点开引用，看到的不只是一个文件，而是文件里的某一段字符区间。术语层则承担查询扩展的职责——用户问“提示注入”，系统需要同时理解 prompt injection、间接注入这些说法。

6.2 检索与打分

检索采用双路召回：关键词路负责 CVE 编号、组件名这类精确匹配，语义路负责口语化描述与跨语言术语。两路结果合并后统一排序，同一分块只保留一条。整条链路如图 5 所示。

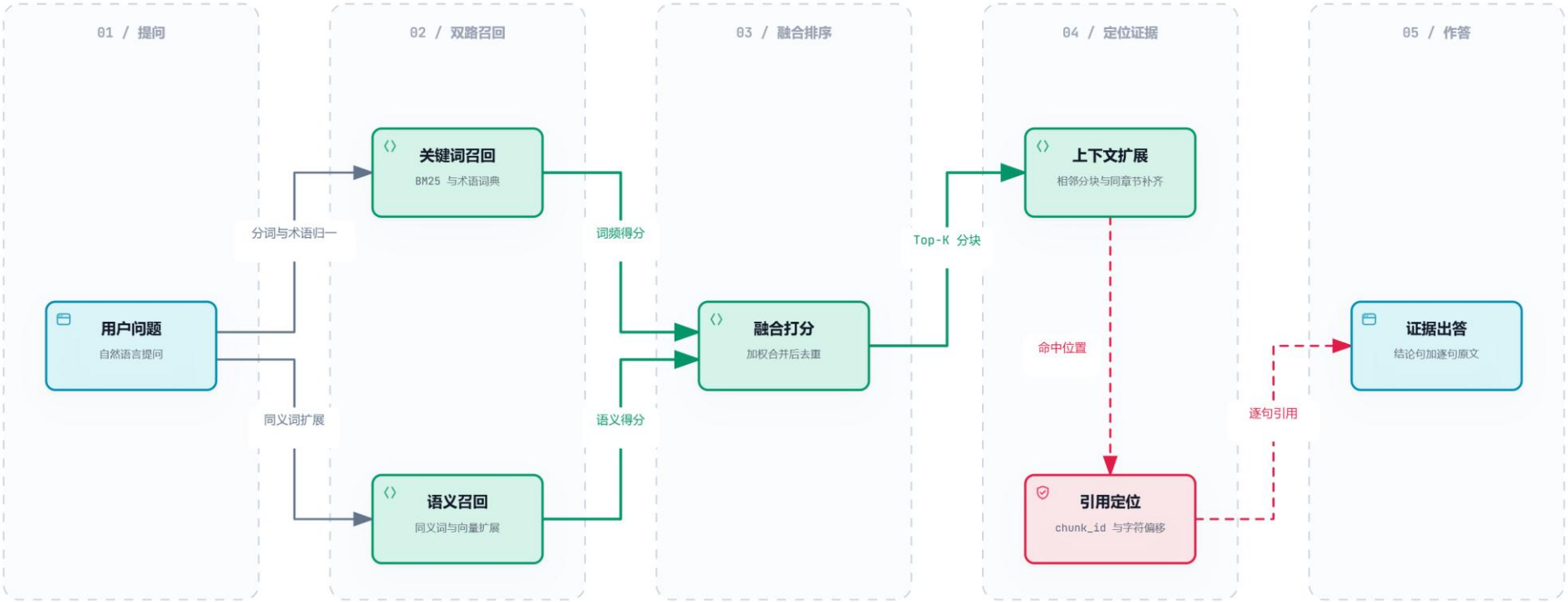

01 / 提问
02 / 双路召回
03 / 融合排序
04 / 定位证据
05 / 作答
用户问题
自然语言提问
分词与术语归一
同义词扩展
关键词召回
BM25 与术语词典
语义召回
同义词与向量扩展
词频得分
语义得分
融合打分
加权合并后去重
Top-K 分块
上下文扩展
相邻分块与同章节补齐
命中位置
引用定位
chunk_id 与字符偏移
逐句引用
证据出答
结论句加逐句原文

图 5　检索与打分链路

打分本身是可复算的：查询词经过分词与同义词扩展后，与文档词频、逆文档频率加权求和，再叠加精确编号命中与组件名匹配的加分。同一份数据、同一个问题，重复运行得到同一个排序，这让评测结果具备可比性，也让“为什么这条排在前面”可以被解释。

上下文扩展是召回之后的补全步骤。命中一个分块后，系统会补齐同章节的相邻分块，避免答案只截到半句话。补齐的范围有上限，防止把整篇文档塞进上下文。

6.3 引用定位

每条引用都携带 chunk_id 与字符偏移，答案里出现的编号必须能在库里取回。评测脚本逐条校验这一点：222 条引用全部回库成功，可追溯率 100%。这个 100% 不是“答对了 100%”，而是“说出口的每一句都有据可查”，两者是不同的指标，报告里分开表述。

引用可追溯带来的直接收益是可复核。评审看到一条结论，可以沿着编号打开原文，确认这句话是不是原文说的、有没有被断章取义。对安全情报这类高 stakes 场景，这比答案“听起来专业”重要得多。

6.4 拒答与澄清

系统对四类情形设定了明确的处理策略，而不是一律硬答。判断顺序是：先看问题是否点名了具体编号，再看语料里是否存在对应记录，最后才进入模糊检索。

表 6-2　拒答与澄清策略

情形	系统行为	对外表现
问题点名了具体 CVE / GHSA，但库中没有	直接拒答并列出缺失编号	“知识库里没有 X 的记录”
问题涉及多个资产候选，指代不清	请求澄清，不擅自选择	列出候选并等待确认
检索到相关事件但证据不足以支撑结论	输出局限说明，明确缺口	“现有资料没有足以支撑的…”
引用 ID 无法回库取回	不写入答案	答案中不出现该引用

第二行是本次修订中补上的一条重要规则。此前的实现里，问题点名一个不存在的编号时，系统会退回到模糊检索，用几条语义相近但无关的文章作答——看起来“有答案”，实际上是答非所问。现在的行为是明确拒答，并告诉用户缺的是哪个编号。这类修正不会让指标变好看，但会让系统更可信。

6.5 问答时序

一次问答从用户提问到答案落地，经过规划、检索、证据读取、答案拼装与引用绑定五个阶段。前端通过 SSE 实时展示每个阶段，用户看到的不是转圈等待，而是系统正在做什么。

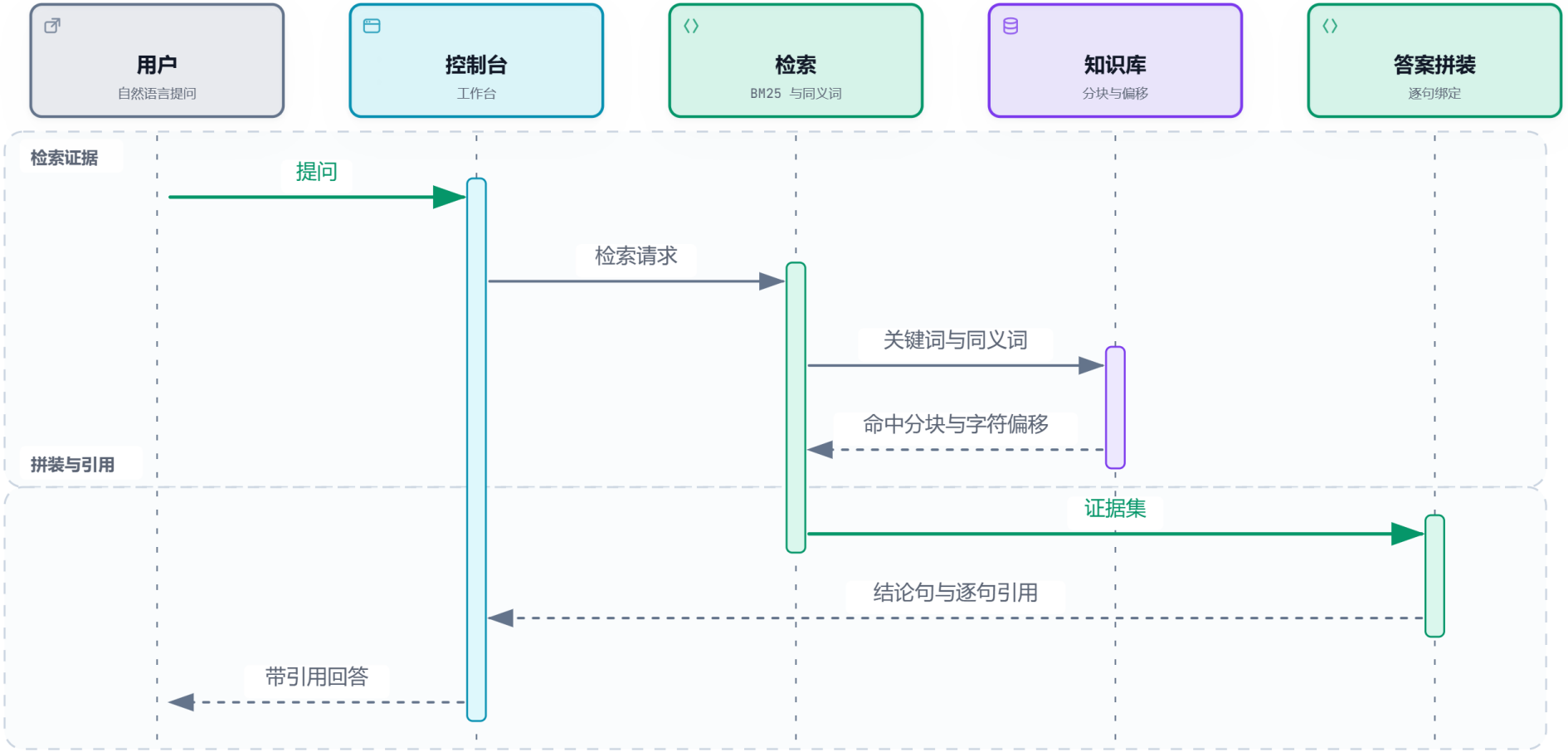


图 6　一次证据问答请求的处理时序

把过程暴露出来的另一个好处是便于定位问题。如果答案不理想，可以沿着时序逐段检查：是检索没召回，还是召回了但排序靠后，还是证据读到了却没有拼进答案。第 11 章的问答指标，正是按这几个环节分别度量的。

第 7 章　处置闭环设计

7.1 状态机

处置闭环要回答的问题很具体：这条风险现在归谁、改到哪一步、改完有没有验证。系统把处置建模成五状态机：待处置、处置中、已修复待复测、已复测关闭、已接受风险。状态之间的迁移有明确前提，不允许跳步。

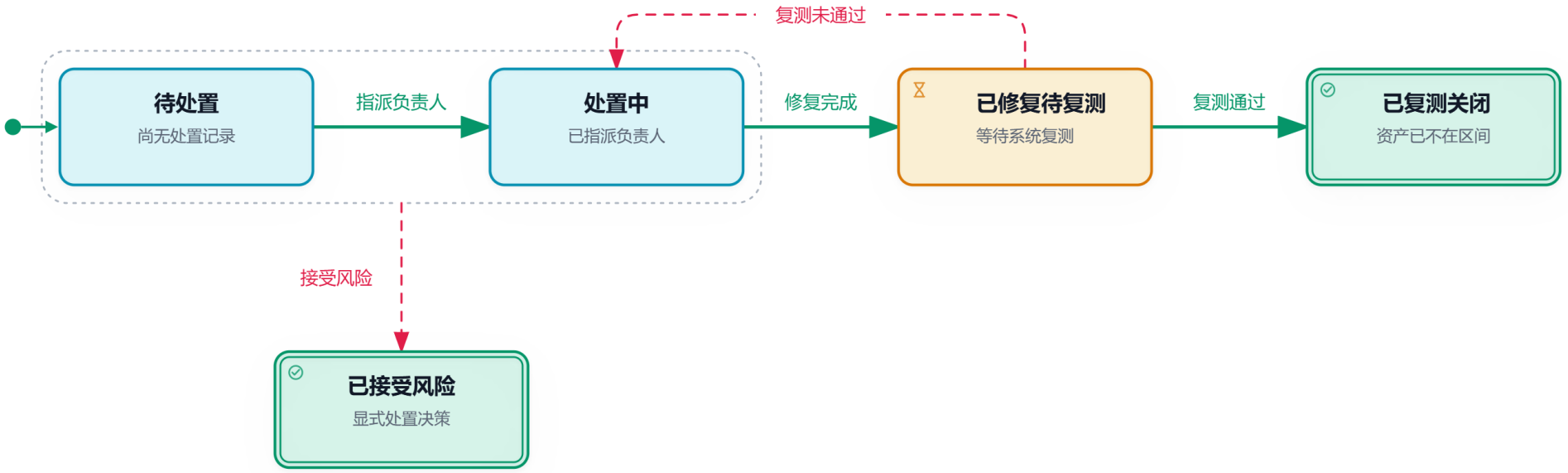

复测未通过
待处置
尚无处置记录
指派负责人
处置中
已指派负责人
修复完成
已修复待复测
等待系统复测
复测通过
已复测关闭
资产已不在区间
接受风险
已接受风险
显式处置决策

图 7　处置闭环状态机

两条规则决定了这套状态机是否可信。**第一，关闭只能由系统复测确认**：人工点“已修复”只是进入待复测，系统重新比对资产版本与受影响区间，确认不再命中才计入关闭。**第二，接受风险是显式决策**：合成资产没有真实补丁路径时，处置人可以选择接受风险，但它单独计数，不冒充“风险已消除”。

7.2 复测机制

复测不是走形式。系统保存处置前后的资产版本，复测时用同一套版本区间判定逻辑重新计算影响状态，并记录复测结论与证据编号。升级到修复版本后重新判定为不受影响的条目，才算真正闭环；版本没变而人工声称“已修复”的，系统不会认可。

7.3 口径与统计

闭环率的口径是：已关闭条目数除以高优先级条目总数。已关闭包含“复测关闭”与“接受风险”两部分，两者在页面上分开列示。已登记处置的条目锁定在分母内，不会因为资产修复而从统计中消失——否则处置越多、分母越小，指标会越算越好看，这不符合审计直觉。

当前快照下，高优先级 55 条，已关闭 53 条（复测关闭 9 条、接受风险 44 条），闭环率 96.36%，未登记处置的条目为 0。需要说明的是，44 条接受风险对应的是合成演示资产，它们的价值在于验证流程可走通，而不代表真实企业环境里的处置能力。

第 8 章　关键技术选型说明

8.1 选型总览

技术选型没有“先进与否”的绝对标准，只有是否匹配约束。本项目的核心约束是：一台普通笔记本可复现、断网也能演示核心能力、任何结论可追溯。基于这三条，我们做了如下取舍。

表 8-1　关键技术选型与取舍

维度	选型	被放弃的方案	理由
存储	SQLite 单库	PostgreSQL / MySQL	零运维、可整库备份、演示机直接可跑
检索	BM25 + 术语扩展	向量数据库 + 嵌入模型	无需下载模型、分数可复算、离线可用
模型	可插拔远端模型 + 三级降级	强制依赖大模型	无密钥时仍能完整作答
前端	原生 HTML/CSS/JS	React / Vue + 构建链	无 node_modules、无打包步骤、可直接审阅
调度	进程内调度器	Airflow / Celery	单机场景下引入调度框架收益为负
部署	Docker + 单进程	Kubernetes	评审环境无法承载集群，容器已足够证明可移植

8.2 为什么用 SQLite

SQLite 在这个场景里不是“简化版数据库”，而是合适的选择。它支持事务、外键、索引与全文检索，单文件即可备份与迁移，读取性能对百万级以内的数据完全够用。更重要的是，评审拿到仓库就能直接打开数据库核对，不需要先装一个数据库服务。

代价是并发写入能力有限。系统的应对方式是把写操作集中在采集与研判两个批次内，读取路径不加锁，页面查询与后台采集可以并行。对单机演示与中小规模部署，这个折中是值得的。

8.3 为什么不用向量检索

向量检索在语义泛化上确实更强，但在本项目的约束下有三个现实问题：嵌入模型需要下载与显存，离线环境不可用；相似度分数依赖模型版本，跨环境不可复算；引用定位仍要回到字符偏移，向量只解决召回不解决取证。

我们选择 BM25 加术语扩展，用领域词典弥补语义泛化。术语层维护 26 个 AI 安全核心术语，在演示机语料上覆盖 83 份文档（换一套语料时该数字随之变化，页面以 `/api/knowledge/terms` 返回为准），把“提示注入 / prompt injection / 间接注入”这类同义表达映射到同一组查询词。在固定评测集上，这套组合的文档命中率达到 97.22%，足以支撑演示与评审场景。

8.4 模型为何可插拔

把模型做成可插拔，是为了避免系统变成“某个模型的演示壳”。接口层只依赖抽象配置：base_url、model、api_key_env 三个字段决定用哪个模型，缺任何一个就自动降级。这样换模型不需要改代码，断网也不会让系统失去作答能力。

8.5 前端为何不用框架

工作台前端约 24 万字节的 JavaScript，全部是原生实现，没有构建步骤。这样做的好处是评审可以直接打开 app.js 阅读逻辑，不需要先理解打包产物；代价是组件复用靠函数封装而非框架机制，代码组织要求更高。对一个以“可审阅、可复现”为目标的参赛作品，这个取舍是合适的。

第 9 章　系统实现细节

9.1 工程结构

仓库按“服务端 / 工具 / 测试 / 文档 / 交付”组织，没有隐藏的生成步骤。服务端代码集中在 app 目录，每个模块对应一类职责，模块之间通过明确的函数签名交互。

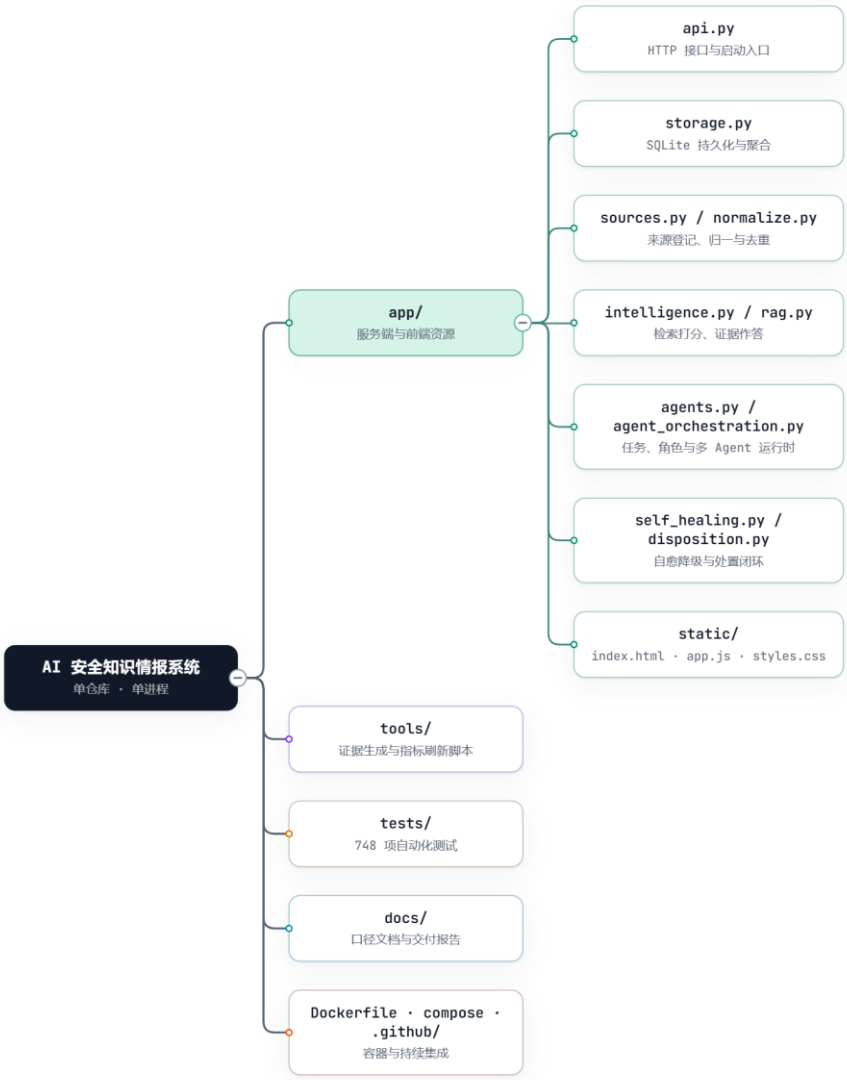


图 8　系统代码结构与模块边界

接口层只做三件事：参数校验、调用编排、结果序列化。业务规则全部下沉到具体模块，因此测试可以直接调用模块函数，不必启动 HTTP 服务。这也是 750 项测试能在四分钟内跑完的原因之一。

9.2 数据模型

数据库共 18 张表，核心是“事实—研判—处置”三段式：events 存外部事实，assessments 存系统判断，assessment_dispositions 存人的处置动作与结果。三段之间用 event_id 与 asset_id 关联，任何一条处置都能追溯到它依据的事件与资产。

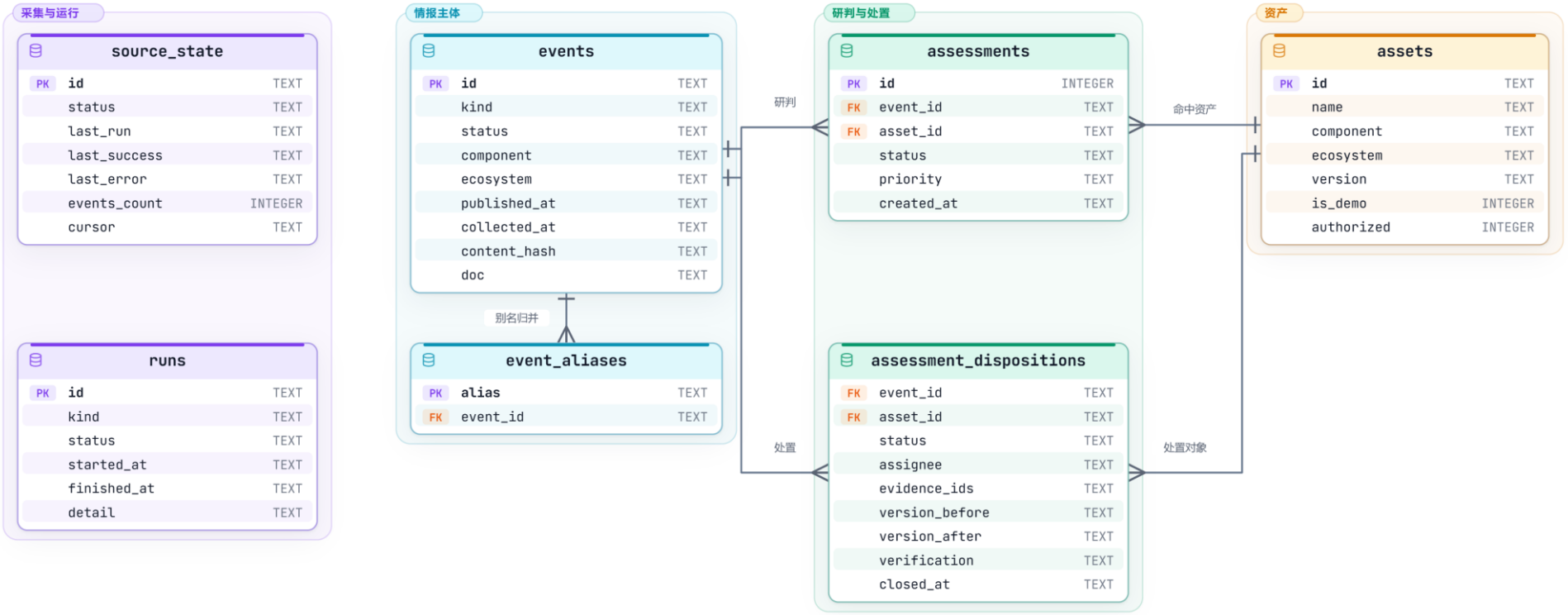


图 9　核心数据模型（8 张主表）

表 9-1　核心数据表

表	行数（快照）	作用
events	705	情报事件主表，含状态、组件、时间与文档指针
event_aliases	233	CVE / GHSA / 厂商编号的别名归并
assets	8	授权资产清单，is_demo 标记演示资产
assessments	196	事件对资产的影响研判结果
assessment_dispositions	55	处置状态、处置人、版本前后与复测结论
runs	1339	五类动作的运行记录
source_state	20	每个来源的健康状态与游标
rag_chunks	849	文档分块与字符偏移

另有三张文档表（rag_documents、rag_document_versions、rag_chunks 的父表关系）支撑原文版本管理，以及一组 agent_* 表记录多 Agent 线程、回合与上下文快照。完整表结构可直接用 SQLite 工具打开数据库查看，无需额外迁移脚本。

9.3 核心代码说明

9.3.1 可复算的双路融合打分

检索打分的核心是下面这段循环：精确编号命中给固定加分，组件名命中给小额加分，其余部分由词频与逆文档频率加权得到。所有常量都写死在代码里，没有随机性，也没有外部模型参与，因此同一份数据重复运行必然得到同一个排序。

代码 1　app/intelligence.py：精确编号优先的融合打分

```
identifiers = {x.casefold() for x in re.findall(
    r"(?:cve-\d{4}-\d{4,}|ghsa-[a-z0-9-]+)", question, re.I)}
for event, terms in zip(events, docs):
    exact = 100.0 if identifiers and any(i in haystack for i in identifiers) else 0.0
    component_bonus = 12.0 if component and component in query_norm else 0.0
    score = 0.0
    for term in query_terms:
        tf = terms.count(term)
        idf = math.log(1 + (len(events) - df.get(term, 0) + .5)
                       / (df.get(term, 0) + .5))
        score += idf * (tf * 2.2) / (tf + 1.2 * (1 - .75 + .75 * len(terms) / max(avg_len,
1)))
    total = exact + component_bonus + score
```

9.3.2 有界的降级判定

降级逻辑的关键在于“什么时候该放弃”。系统只在连续失败达到阈值时才降级，并且把失败原因分类记录下来，而不是简单写一个 unknown。这样运维看到降级记录时，能立刻知道是限流、超时还是格式变化。

代码 2　app/self_healing.py：连续失败后的有界动作

```
if streak < CONSECUTIVE_FAILURE_THRESHOLD:          # 默认 3 次
    return None
current = state or {}
failure_class = classify_failure(current.get("status"), current.get("last_error"))
spec = sources.get(source_id)
backups = [item for item in (spec.backup_sources if spec else ()) if item]
if backups:
    return {
        "source_id": source_id,
        "action": "switch_backup",
        "backup_sources": backups,
        "failure_class": failure_class,
        "consecutive_failures": streak,
    }
```

9.3.3 可观测的流式帧

流式输出的每一帧都回答四个问题：做了什么、拿到什么、产出什么、有什么影响。这个契约写在一个函数里，所有阶段共用，保证前端不需要为不同阶段写不同的渲染分支。

代码 3　app/streaming.py：统一的阶段帧契约

```
payload: dict[str, Any] = {
    "seq": seq.next(),
    "phase": phase,
    "phase_label": PHASE_LABELS.get(phase, phase),
    "role": role,
    "role_label": ROLE_LABELS.get(role, role),
    "title": title,
    "action": action,
    "obtained": obtained,
    "output": output,
    "effect": effect,
    "status": status,
    "refs": refs or [],
}
```

9.4 接口设计

系统共 73 个接口路径，其中 40 个 GET、31 个 POST、1 个 PUT、1 个 DELETE，按能力分为九组：采集与运行、事件与检索、知识与图谱、研判与资产、处置、智能体、系统健康、评测、流式输出。所有写接口都经过同源校验中间件，跨站请求直接拒绝。

表 9-2　接口分组

接口组	代表路径	说明
采集与运行	/api/collect、/api/runs	触发采集、查询运行记录
事件与检索	/api/events、/api/rag/search	事件浏览与分块检索
知识与图谱	/api/knowledge/graph、/api/knowledge/terms	图谱节点、术语字典
研判与资产	/api/assessments、/api/assets	资产清单与影响研判
处置	/api/dispositions、/api/dispositions/metrics	处置状态与闭环指标
智能体	/api/agent/tasks、/api/agent/multi/execute	任务 DAG 与多 Agent 执行
系统健康	/api/system/health、/api/system/model-chain	自愈状态与模型链路
评测	/api/competition/scorecard、/api/evaluation/b	赛题指标与人工评测

流式输出	/api/stream/answer、/api/stream/pipeline	SSE 实时过程

9.5 一致性与并发

单库架构下，一致性靠事务与幂等写入保证。采集写入以内容哈希为主键，重复抓取不会产生重复事件；研判与处置各自在事务内完成，失败整体回滚。读取路径不加写锁，页面查询与后台采集可以并行。

并发压力主要出现在“页面查询图谱”与“后台采集”同时进行时。图谱接口对节点数量设了上限，超出部分截断并标注 truncated，而不是把整库拉到内存里。这类边界处理在报告里不显眼，但它决定了系统在长时间运行后是否还站得住。

第 10 章　功能模块实现效果

系统以单页工作台承载全部功能，左侧三个页签对应三条主线：任务工作台负责采集与研判，知识图谱负责知识组织与证据检索，赛题指标负责能力自证。三页共享同一份数据，不存在演示数据与统计口径不一致的问题。

10.1 任务工作台

任务工作台是主入口。顶部给出当前风险与资产的总体态势，中部是优先处置队列，下方按“抓取—整理—补证—研判”四步展开任务链路。每一项数字都可以点开看到明细，例如“受影响资产 112”点进去就是逐条研判记录。

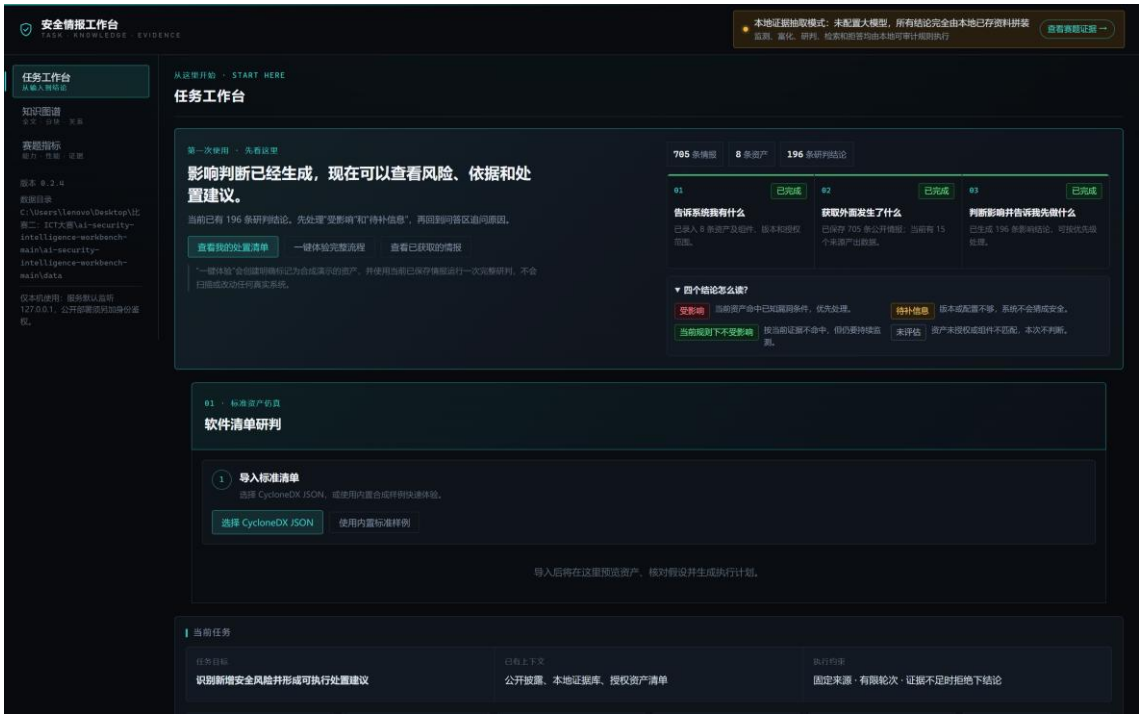


图 10　任务工作台首屏

10.2 知识图谱

知识图谱页把术语、事件与关系放在同一视图里。左侧是 26 个 AI 安全核心术语，每个术语带覆盖文档数与证据样例；下方按“提示注入”“越狱”“数据投毒”等主题分组列出相关事件，点开任意一条可以跳到原文的字符位置。

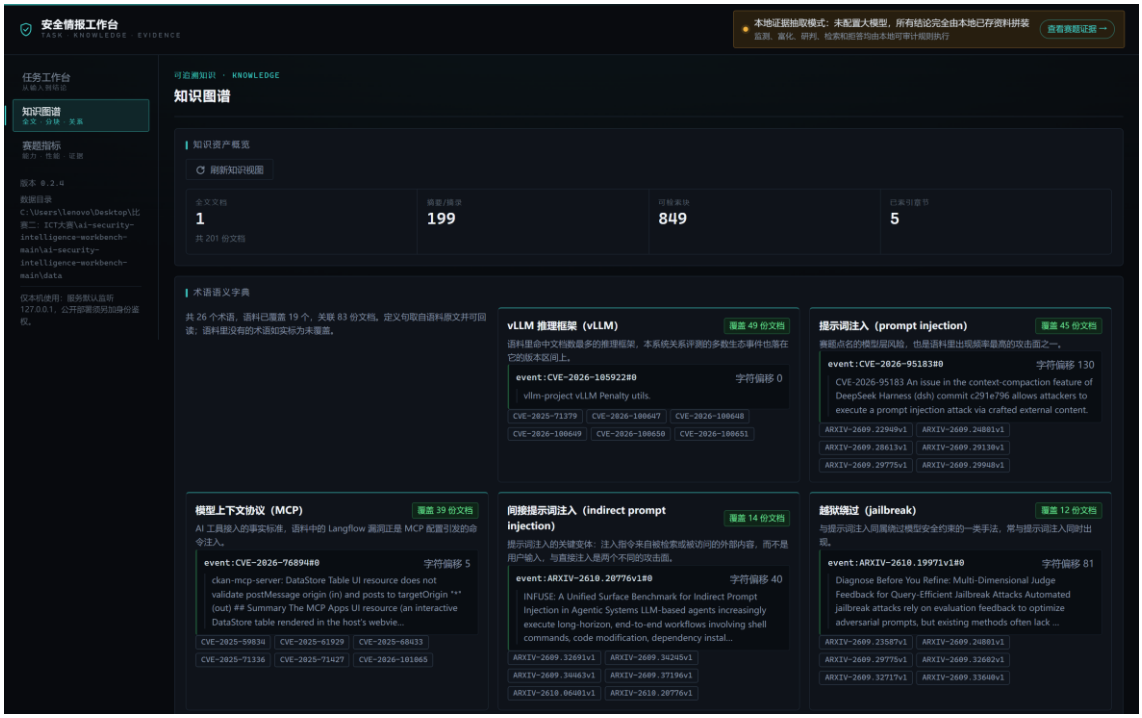


图 11　知识图谱与术语证据

10.3 赛题指标

赛题指标页是系统的自证页面。每一项能力卡都标注“已验证”或“待积累”，并给出计算口径。需要强调的是，这些数字全部来自接口实时读取，没有任何一项是手工填写的常量。

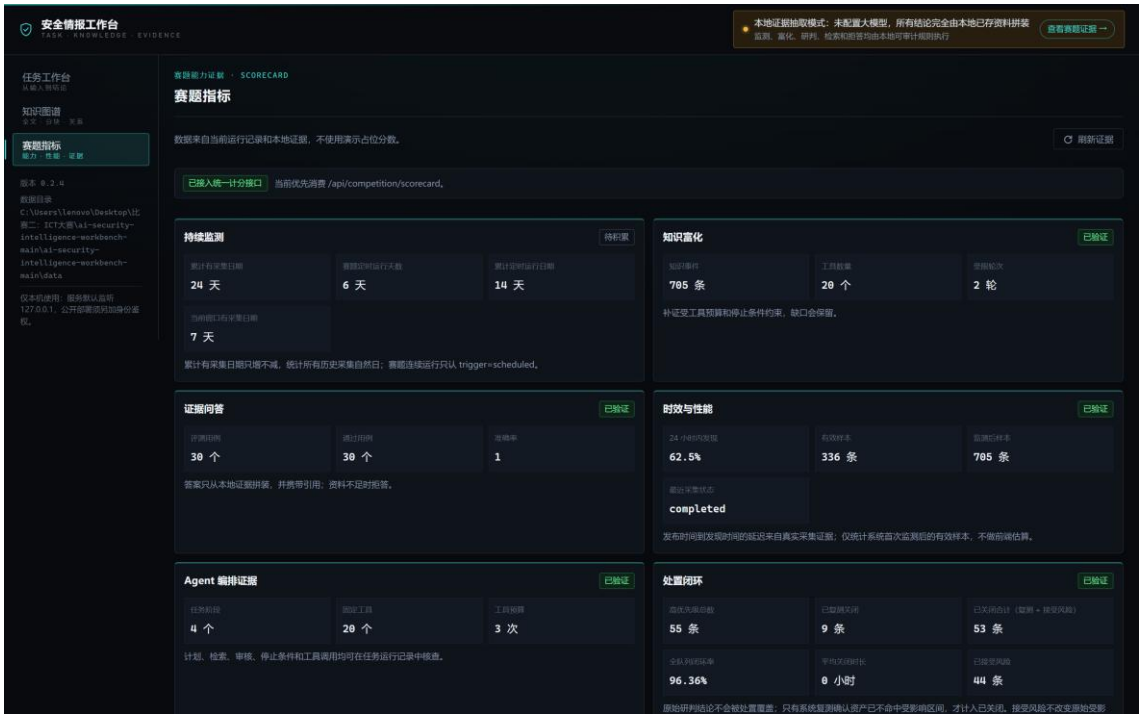


图 12　赛题指标与能力证据

10.4 模块效果对照

表 10-1　模块效果与证据位置对照

模块	可见结果	证据位置
情报监测	20 个来源状态、15 个持续产出、运行记录 1339 条	赛题指标 • 持续监测 / 来源健康度
知识富化	CVSS 480 条、POC 96 条、论文关联 162 条	知识图谱 • 事件详情
证据问答	文档命中 97.22%、引用可追溯 100%	赛题指标 • 证据问答
关系评测	P 1.000 / R 0.734 / F1 0.8466	赛题指标 • 关系与多跳评测
处置闭环	55 条高优先级，53 条关闭，闭环率 96.36%	赛题指标 • 处置闭环
工程交付	750 项测试通过，CI 三项作业全绿	赛题指标 • 工程化 / CI 证据

第 11 章　性能测试结果与分析

11.1 测试环境与方法

全部性能读数来自同一台演示机上的同一份数据库，测试脚本直接调用接口，不经过浏览器渲染，以避免前端开销干扰。时效指标使用来源自带发布时间与系统入库时间之差，只统计首次监测之后发布的样本，历史回填数据单独排除。

表 11-1　测试环境

项目	配置
运行环境	Windows 11 • Python 3.12 • 单进程 uvicorn

数据库	SQLite 单文件，快照 15.1 MB
数据规模	705 条事件 / 1059 份文档 / 849 个分块
测试方式	脚本直连接口，逐条计时，取中位数与 P95
评测集	固定 50 题问答集 + 36/38 条预期文档与术语 + 94 条关系金标准

11.2 采集时效

漏洞类情报的中位采集延迟为 7.34 小时，样本 244 条，落在赛题要求的 12 小时档位之内。分来源看，GitHub Advisory 的中位延迟为 2.20 小时，NVD 为 8.27 小时，差异来自两个库的发布节奏，报告如实保留，不做合并美化。

表 11-2　采集时效分布

口径	样本	中位延迟	6 小时内	24 小时内
漏洞类（赛题口径）	244	7.34 小时	44.7%	73.8%
混合口径（含学术，附录备查）	336	14.43 小时	32.4%	62.5%

混合口径的中位延迟明显更高，原因是学术论文的发布到收录周期天然长于漏洞公告。把两类混在一起统计会掩盖漏洞监测的真实水平，因此报告以漏洞类为主口径，混合口径仅作附录备查。

11.3 证据问答质量

问答评测拆成四个互相独立的维度：预期文档是否被引用、预期术语是否在答案中出现、引用能否回库取回、该拒答的问题是否拒答。四个维度分别计算，不合成单一分数，避免“平均分好看但某一项很差”的情况。

表 11-3　证据问答评测结果

维度	结果	分母	说明
文档命中	97.22%	36 条预期文档	35 条被正确引用
要点命中	92.11%	38 个预期术语	35 个在答案中逐字出现
引用可追溯	100%	222 条引用	全部可按 chunk_id 回库取回
拒答一致	84%	50 道题	42 道与 should_refuse 标签一致
响应时延	P50 459 ms / P95 640 ms	同一批 50 题	本地检索与答案拼装的总耗时

时延数据说明一件事：整套问答不依赖外部网络，P95 仍在 640 毫秒以内，现场演示时不会出现长时间等待。这对答辩场景很重要——评审不会为一个转圈等三十秒。

11.4 关系抽取评测

关系抽取采用人工金标准逐条判定。94 条可判定样本中，系统抽出 69 条正确关系，没有误报，漏掉 25 条。精确率 1.000 意味着系统不会凭空造关系；召回率 0.734 的缺口主要来自语料中确实不存在可核验引用的文档对。

表 11-4　关系抽取评测

指标	数值	说明
Precision	1.000	TP 69 / (TP 69 + FP 0)
Recall	0.734	TP 69 / (TP 69 + FN 25)
F1	0.8466	精确率与召回率的调和平均
可判定样本	94	58 条源头缺少可比事实，排除在分母外

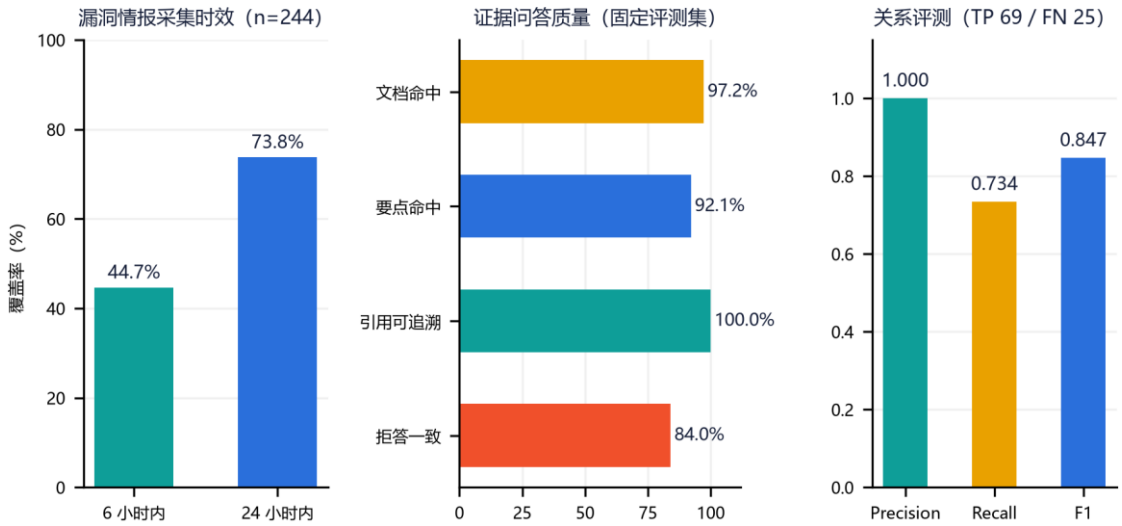


图 13　三项核心结果对照

11.5 工程验证

工程侧的验证不看说法看证据：测试全量运行、持续集成三项作业、容器构建、数据库回滚，每一项都有可打开的运行记录。

表 11-5　工程验证结果

验证项	结果	证据
自动化测试	750 项通过 / 0 跳过	pytest 全量运行，耗时约 4 分钟
持续集成	3 / 3 作业成功	容器构建、回滚验证、测试
容器镜像	构建成功并启动	python:3.12-slim + 健康检查
数据库回滚	备份与恢复路径可用	CI rollback 作业
代码规模	服务端 30 余个模块	app 目录，按职责拆分

11.6 结果分析

把三组结果放在一起看，可以得到一个相对清晰的判断：系统的强项在**证据链的完整性**——引用可追溯 100%、关系误报 0、测试零跳过，这三项说明系统不会为了好看而编造内容。相对薄弱的是**召回**，无论是关系抽取的 0.734 还是拒答一致性的 84%，背后都是同一个原因：语料本身覆盖不全。

这不是靠调参能解决的问题。要提升召回，需要扩充互相引用的论文语料、补充更多厂商公告，而不是放松判定阈值把不确定的东西也算作命中。系统的选择是保留缺口、如实展示，把“不知道”和“知道”分得清清楚楚。

第 12 章　创新点与展望

12.1 创新点

本系统的创新不在单点算法，而在把“证据约束”贯彻到了整条链路。具体可以归纳为四条。

- **证据优先的答案组织**：答案先给结论句，再附逐字原文与 chunk 编号；引用取不回就不写进答案，无法支撑时明确拒答。这把“可信”从形容词变成了可校验的机制。
- **可复算的检索与判定**：双路召回融合打分全部是确定性计算，同一份数据重复运行得到同一结果；机器指标与人工指标分列，不合成单一分数。
- **可审计的处置闭环**：处置记录带处置人、版本前后与复测结论；关闭只能由系统复测确认，接受风险单独计数。这让“风险已消除”这句话有了可验证的依据。
- **失败可见的自愈机制**：来源级状态机、连续失败降级、冷却后自动归队，并保留降级期间的历史数据与失败分类。系统不会把失败伪装成成功。

12.2 局限

报告如果不写局限，可信度反而会打折。以下四条是系统当前明确的短板。

- **语料规模有限**：705 条事件对一个真实安全团队远远不够，关系召回与多跳推理的缺口主要来自数据量。
- **语义泛化弱于向量检索**：BM25 加术语扩展在精确词上表现好，但对口语化、跨段落的语义关联仍有明显上限。
- **资产数据为合成演示数据**：8 条资产用于验证流程可走通，不能代表真实企业环境下的处置能力。
- **单机架构的并发上限**：SQLite 与单进程足以支撑演示与中小规模使用，但面对高并发写入需要改造存储层。

12.3 展望

下一步的改进方向与局限一一对应：扩充互相引用的论文与厂商公告语料以提升召回；在保留确定性打分的前提下引入可选的向量召回作为补充路径；接入真实资产清单与 CMDB，把处置闭环从演示推向生产；在存储层预留 PostgreSQL 适配接口，为规模化部署留出空间。

更长远地看，这套系统的价值不在于“又做了一个安全大屏”，而在于验证了一种工程方法：在模型能力不确定、数据来源不可控的条件下，用证据约束与确定性计算把系统做成可交付、可复核、可复现的工程件。这条方法论可以迁移到其他需要“结论必须站得住”的场景。

附录 A　指标口径

附录列出报告中出现的全部关键指标及其计算方式，便于复算与核对。所有指标均来自接口或数据库，不接受人工填报。

表 A-1　指标口径

指标	定义	分母 / 来源
采集时延	来源发布时间到系统入库时间之差的中位数	首次监测后发布的样本
6 / 24 小时覆盖率	在上述时延内完成采集的样本占比	漏洞类 244 条
文档命中	预期文档出现在答案引用中的比例	36 条预期文档
要点命中	预期术语逐字出现在答案中的比例	38 个预期术语
引用可追溯	引用 ID 能按 chunk_id 回库取回的比例	222 条引用
拒答一致	拒答行为与 should_refuse 标签一致的比例	50 道题
关系精确率	TP / (TP + FP)	人工金标准 94 条可判定样本
闭环率	已关闭 / 高优先级总数	55 条高优先级
连续运行天数	7 天窗口内有定时运行的最大连续天数	仅统计 trigger=scheduled

附录 B　复现步骤

在一台装有 Python 3.12 的机器上，按以下步骤可以在五分钟内复现系统与报告中的关键读数。

代码 4　本地复现命令

```
# 1. 安装依赖并启动服务
python -m pip install -r requirements.txt
python -m uvicorn app.api:app --host 127.0.0.1 --port 8000

# 2. 浏览器打开工作台
http://127.0.0.1:8000/

# 3. 复算赛题指标
curl http://127.0.0.1:8000/api/competition/scorecard

# 4. 全量测试
python -m pytest -q

# 5. 容器方式（可选）
docker compose up --build
```

表 B-1　复现检查清单

检查项	预期结果
首页可访问	HTTP 200，显示任务工作台
健康检查	/api/health 返回 ok
赛题指标	闭环率 96.36%，测试与 CI 证据可查
问答	任意问题都能看到引用或明确的拒答理由
测试	750 passed，0 skipped